

Streaming output and events

When the expected answer is long and we want to immediately start returning output, we can use live token streams and event callbacks to see progress instantly.

 The `AgentWorkflow` also supports streaming. The `AgentWorkflow` can be streamed like any other `Workflow`. This works by using the handler that is returned from the workflow. The stream returns a variety of event types as the workflow executes, and we can select which ones to handle.

- If we want to stream the LLM output, we can use the `AgentStream` events, which contain a `delta` of the new output each time
- `AgentInput` events will tell us which agent is running (our current workflow just has one agent)
- `AgentOutput` events will tell us what the agents returned, including which tools they called
- `ToolCall` and `ToolCallResults` will track tools as they are called and their outputs

In this example we're handling the `AgentStream` events. We can tell our handling is working because the output will appear in chunks as we run the cell, rather than appearing all at once.

! Note: Run the hidden cells below before running the rest of the code. !

3 hidden cells

```

import os

# Install the correct package for Tavily tool spec
%pip install llama-index-tools-tavily-research

from llama_index.core.agent.workflow import AgentWorkflow, AgentStream
from llama_index.tools.tavily_research.base import TavilyToolSpec # <-- Correct
import

# Define your LLM instance before using it.
from llama_index.llms.openai import OpenAI
API_BASE = "https://api.openai.com/v1" # Define the API_BASE variable

API_KEY = "sk-proj-vVoenQIQW_Ag_9o_H0R4uHcMrQ1B4uNs0-
_U9r8coI5M4D5FArggiGxNwWA9JMs9gv4WQ4Ic_TT3B1bkFJSu86H84B9HpeK0ZzE-cUEsbshU-
rdSI_7aTw11Lflv1KJcGd9SyoN4yxM6NKMT9R9JAY5DTVMA"

llm = OpenAI(model="gpt-4o-mini", api_key=API_KEY, api_base=API_BASE)

# A search tool provides access to current information.
tavily_tool = TavilyToolSpec(
    api_key=os.environ["TAVILY_API_KEY"]
)

# Create a configured agent, not an empty Workflow().
workflow = AgentWorkflow.from_tools_or_functions(
    tools_or_functions=tavily_tool.to_tool_list(),
    llm=llm,
    system_prompt=(
        "You are a helpful assistant. "
        "Use web search for current weather questions. "
        "Include the observation time and source when available. "
        "Do not invent weather information."
    ),
)

# Do not await yet: keep the handler to consume streaming events.
handler = workflow.run(
    user_msg="What is the current weather in Saskatoon?"
)

async for event in handler.stream_events():
    if isinstance(event, AgentStream):
        # delta contains only the newly generated text chunk.
        print(event.delta, end="", flush=True)

# Retrieve the completed result and surface any execution error.

```

```
response = await handler
print() # Finish the line after streaming
```

Requirement already satisfied: llama-index-tools-tavily-research in /home/repl/.venv-user/lib/python3.12/site-packages (0.6.0)
Requirement already satisfied: llama-index-core<0.15,>=0.13.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-tools-tavily-research) (0.14.24)
Requirement already satisfied: tavily-python>=0.2.4 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-tools-tavily-research) (0.8.3)
Requirement already satisfied: aiohttp<4,>=3.8.6 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (3.12.15)
Requirement already satisfied: aiosqlite in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (0.22.1)
Requirement already satisfied: banks<3,>=2.3.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (2.5.1)
Requirement already satisfied: dataclasses-json in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (0.6.7)
Requirement already satisfied: deprecated>=1.2.9.3 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (1.2.18)
Requirement already satisfied: dirtyjson<2,>=1.0.8 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (1.0.8)
Requirement already satisfied: filetype<2,>=1.2.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-tools-tavily-research) (1.2.0)
Requirement already satisfied: fsspec>=2023.5.0 in /home/repl/.venv-user/lib/python3.12/site-packages (from llama-index-core<0.15,>=0.13.0->llama-index-

Let's see what's happening in more detail by turning on more logging and running it again.

```

from llama_index.core.agent.workflow import (
    AgentInput,
    AgentOutput,
    ToolCall,
    ToolCallResult,
    AgentStream,
)

handler = workflow.run(user_msg="What is the weather in Saskatoon?")

async for event in handler.stream_events():
    if isinstance(event, AgentInput):
        print("Agent input: ", event.input) # Current input messages
        print("Agent name:", event.current_agent_name) # Current agent name
    elif isinstance(event, AgentOutput):
        print("Agent output: ", event.response) # Current full response
        print("Tool calls made: ", event.tool_calls) # Selected tool calls, if any
        print("Raw LLM response: ", event.raw) # Raw llm api response
    elif isinstance(event, ToolCallResult):
        print("Tool called: ", event.tool_name) # Tool name
        print("Arguments to the tool: ", event.tool_kwargs) # Tool kwargs
        print("Tool output: ", event.tool_output) # Tool output

```

```

Agent input: [ChatMessage(role=<MessageRole.SYSTEM: 'system'>, additional_kwargs={}, blocks=[TextBlock(block_type='text', text='You are a helpful assistant. Use web search for current weather questions. Include the observation time and source when available. Do not invent weather information.')] ), ChatMessage(role=<MessageRole.USER: 'user'>, additional_kwargs={}, blocks=[TextBlock(block_type='text', text='What is the weather in Saskatoon?')])]
Agent name: Agent
Agent output: assistant:
Tool calls made: [ToolSelection(tool_id='call_SpHcIXMxH1jzyTcCtwMGa89U', tool_name='search', tool_kwargs={'query': 'current weather Saskatoon', 'max_results': 3})]
Raw LLM response: {'id': 'chatcmpl-EPGwMosjZA7760jfcFZgGmj4Y0V7z', 'choices': [{'delta': {'content': None, 'function_call': None, 'refusal': None, 'role': None, 'tool_calls': None}, 'finish_reason': 'tool_calls', 'index': 0, 'logprobs': None}], 'created': 1789692026, 'model': 'gpt-4o-mini-2024-07-18', 'object': 'chat.completion.chunk', 'moderation': None, 'service_tier': 'default', 'system_fingerprint': 'fp_0e26109f93', 'usage': None, 'obfuscation': '9JCCCifk4qbU'}

```

We can see the agent accepting input, picking the web search tool, calling it with appropriate arguments, and returning the output.

